arXiv:2306.10785v1 [cs.SD] 19 Jun 2023

MULTITRACK MUSIC TRANSCRIPTION WITH A TIME-FREQUENCY PERCEIVER

Wei-Tsung Lu, Ju-Chiang Wang, and Yun-Ning Hung

SAMI, ByteDance, Mountain View, CA, USA

{weitsung.lu, ju-chiang.wang, yunning.hung}@tiktok.com

ABSTRACT

Multitrack music transcription aims to transcribe a music audio input into the musical notes of multiple instruments simultaneously. It is a very challenging task that typically requires a more complex model to achieve satisfactory result. In addition, prior works mostly focus on transcriptions of regular instruments, however, neglecting vocals, which are usually the most important signal source if present in a piece of music. In this paper, we propose a novel deep neural network architecture, *Perceiver TF*, to model the time-frequency representation of audio input for multitrack transcription. Perceiver TF augments the Perceiver architecture by introducing a hierarchical expansion with an additional Transformer layer to model temporal coherence. Accordingly, our model inherits the benefits of Perceiver that posses better scalability, allowing it to well handle transcriptions of many instruments in a single model. In experiments, we train a Perceiver TF to model 12 instrument classes as well as vocal in a multi-task learning manner. Our result demonstrates that the proposed system outperforms the state-of-the-art counterparts (e.g., MT3 and SpecTNT) on various public datasets.

***Index Terms*—** Time-frequency, Perceiver, automatic music transcription, multi-task learning, random-mixing augmentation.

1. INTRODUCTION

Automatic music transcription (AMT) is a Music Information Retrieval (MIR) task that aims to transcribe a music audio input into a sequence of musical notes where each note contains attributes of onset, pitch, duration, and velocity. The output is typically delivered in the format of MIDI. In a multitrack setting, an AMT system should identify every instrument that is present in the input, and estimate the associated notes accordingly into a channel of the MIDI output. Ideally speaking, using the identified instrument for each corresponding channel, the synthesized audio mixture from the output MIDI should resemble the original input audio in a musically plausible way.

Although recent years have seen significant progress using deep learning techniques [1, 2], our analysis and review indicate that two major challenges have not yet been addressed effectively: *model scalability* and *instrument discrimination*. Multitrack AMT is generally regarded as a very challenging task. The number of commonly used instruments can be up to 100. Among them, musical notes of regular instruments like guitar, violin, and synthesizers are difficult to characterize due to their tremendous variations in timbre, expressivity, and playing techniques. Other than that, vocals, which usually are the most predominant instrument if present, vary their timbre and pitch to convey lyrics and expressions. To handle all instruments simultaneously, it requires better *model scalability*. Our observations on existing multitack AMT systems reveal that they oftentimes result in many false positive notes for popular pitched instruments like piano and guitar. For instance, notes of string ensemble are massively captured by piano. This might be because the system does not provide clear timbre-dependent features or it is not robust to timbral variations across different instruments. We believe this problem can be mitigated if the system can *discriminate* each instrument source from the mixture while making inference.

To address model scalability, we propose *Perceiver TF*, which is an augmented variant of the Perceiver [3]. Perceiver has been well-known for its better scalability in the Transformer family to tackle high-dimensional data input. In this work, we adopt spectrogram for audio input, with T and F representing the lengths of the time- and frequency-axes, respectively. For multitrack AMT, capability to model the timbre-dependent pitches of multiple instruments is crucial, so more comprehensive operations are needed to capture useful features along the high-resolution frequency axis. Recently, the SpecTNT architecture [4] was proposed for this purpose and achieved state-of-the-art performance in vocal melody extraction (a sub-task of AMT). SpecTNT consists of two Transformers in a hierarchical structure, where the lower-level Transformer performs self-attention directly on the spectrum of a frame. However, such design leads to a cubic complexity of attention computation, i.e., $\mathcal{O}(TF^2 + T^2)$, limiting its expandability for more complex tasks. To this end, we conceive a non-trivial combination of Perceiver and SpecTNT: expanding Perceiver to be hierarchical. The resulting *Perceiver TF* takes advantage of the cross-attention to extract spectral features into a latent bottleneck for each frame, and adds an additional Transformer for self-attention along the time axis, overall resulting in a quadratic complexity of $\mathcal{O}(TF+T^2)$. Since F is typically large, this complexity reduction is significant, allowing the model to handle more instruments simultaneously.

To address *instrument discrimination*, we adopt the *random-mixing* augmentation technique learned from music source separation (MSS) [5, 6], which aims to separate each instrument stem from the input audio mixture [7]. Moreover, we train our AMT model in a multi-task learning fashion, with each sub-task modeling the transcription of an instrument. This multi-task design along with the random-mixing technique allows more flexibility to train with enormous amount of augmented training samples. Our strategy differs from previous works that jointly train the AMT task with instrument recognition [8] or MSS [9] to help inform the model of instrument-dependent features. To our knowledge, little work has been done using random-mixing technique to improve multitrack AMT.

2. RELATED WORK

Multi-instrument AMT has been explored in several previous works. Wu et al. [10] and Hung et al. [8] trained a transcription model with related tasks in a multi-task learning fashion. Tanaka et al. used clustering approaches to separate transcribed instruments [11], while Cheuk et al. used unsupervised learning techniques to im-

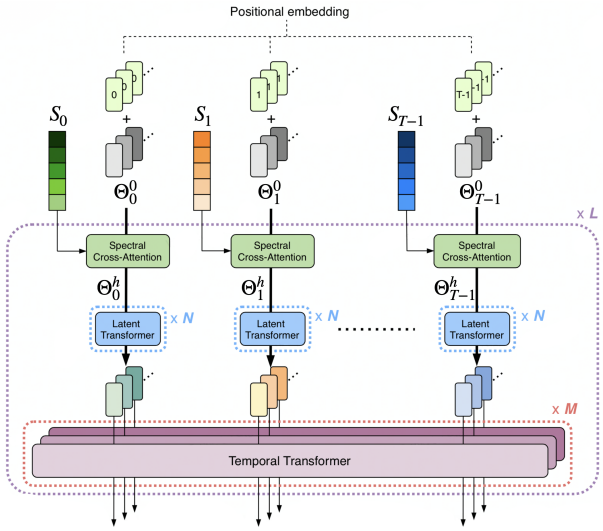

Fig. 1. The block diagram of the Perceiver TF module. Positional embedding is first added to the latent arrays, denoted as Θ_t^0. The Spectral Cross-Attention module projects the spectral input S_t into Θ_t^h, followed by the Latent Transformer module. The Temporal Transformer processes Θ_t^h of all time steps to model the temporal coherence. The details are explained in Section 3.2.

prove transcription on low-resource datasets [1, 12]. These prior examples demonstrated that models based on the pianoroll representation are able to capture instrument-dependent onset, pitch, and duration of notes. Different from the pianoroll approach, Gardner et al. [2] created a new paradigm that proposes a sequence-to-sequence model, called MT3, to tackle multitrack AMT. They trained a standard encoder-decoder Transformer to model multitrack MIDI tokens from audio, and demonstrated state-of-the-art performance on several public datasets.

By contrast, vocal transcription is usually treated as an independent task in the literature, even though it shares the same goal of AMT. Due to the lack of training data, few works focused on transcribing note-level outputs from polyphonic music audio. Recently, Wang et al. released a human annotated dataset including 500 Chinese songs [13]. They provide a CNN based model (EFN) for a baseline of the task. In [14], a teacher-student training scheme is proposed to utilize pseudo labels derived from F0 estimations of vocal. Lately, [15] proposed a vocal transcription system that requires an MSS as front-end. In this work, we propose a unified framework that combines vocal and multi-instrument transcriptions, and it does not rely on pre-trained modules such as an MSS front-end.

3. METHODOLOGY

In this work, we adopt the pianoroll approach instead of the sequence-to-sequence (seq-to-seq) approach for two major reasons. First, it is easier to manipulate the loss computation to learn from partially labeled data. For example, it is non-trivial to train a seq-to-seq model that joints a vocal transcription dataset where the MIDI ground truth of accompaniments is not available. Second, the inference time complexity of seq-to-seq depends on the number of notes (tokens) due to the auto-regressive nature. If the audio input contains many instruments with complex, dense polyphonic notes, the inference will be very slow. Although our proposed model is also a Transformer-oriented architecture, we focus on the encoder part to predict the pianoroll directly.

The following sections explain the proposed model architecture (Sections 3.1 – 3.3) and the random-mixing augmentation technique (Section 3.4). Our model consists of three sub-modules: convolutional module, Perceiver TF module, and output module. The input spectrogram is first passed through the convolution module for local feature aggregation. Then, the perceiver TF module, which includes multiple Perceiver TF blocks, extracts the features and outputs the temporal embeddings at each time step. Lastly, the output module projects the temporal embeddings into desired dimensions for pianoroll outputs.

3.1. Convolutional Module

Using convolutional neural network (CNN) as the front-end of Transformer-based models has became a common design choice in speech recognition pipeline [16]. Previous works have also found that the CNN front-end plays an crucial role in SpecTNT and MIRTransformer for many MIR tasks [4, 17, 18, 19]. Following this practice, we stack multiple residual units [20] with average pooling to reduce the dimensionality of the frequency axis. We denote the resulting time-frequency representation as $\mathcal{S} = [S_0, S_1, \ldots, S_{T-1}] \in \mathbb{R}^{T \times F \times C}$, where T, F, and C represent the dimensions of time, frequency, and channel, respectively.

3.2. Perceiver TF Module

A conventional Perceiver architecture contains two major components [3]: (i) a cross-attention module that maps the input data and a latent array into a latent array; (ii) a Transformer tower that maps a latent array into a latent array. Upon this structure, our design principle to expand Perceiver is twofold. (1) We consider the spectral representation of a time step, S_t, is pivotal to carry the pitch and timbral information, so it serves as the input data for the cross-attention module to project the spectral information into a latent array for the time step t. Each latent array is responsible for extracting the local spectral features. (2) Having a sequence of latent arrays of different time steps, we need a Transformer to exchange the local spectral information along the time axis to learn their temporal coherence.

The Perceiver TF architecture is illustrated in Fig. 1. A Perceiver TF block contains three Transformer-style modules: *spectral cross-attention*, *latent Transformer*, and *temporal Transformer*, which are responsible for modeling the spectral, channel-wise, and temporal information, respectively. Each of them includes the attention mechanism and a position-wise feed-forward network.

The *spectral cross-attention* (SCA) module operates directly on an input spectral representation S_t and projects it into the Key (K) and Value (V) matrices. Unlike the traditional Transformer, the cross-attention module in Perceiver maps a latent array into the Query (Q) matrix and then performs the QKV self-attention accordingly. We follow the Perceiver design to initialize a set of K learnable latent arrays $\Theta^0 \in \mathbb{R}^{K \times D}$, where K is the index dimension, and D is the channel dimension. Then, we repeat Θ^0 for T times and associate each to a time step t, which is then denoted as Θ_t^0, such that $\Theta_0^0 = \Theta_1^0 = \ldots \Theta_{T-1}^0$, meaning that all latent arrays are from the same initialization across the time axis. This Θ_t^h plays an important role of carrying the spectral information from the first Perceiver TF block throughout the entire stack of blocks. The query-key-value (QKV) attention of our SCA of the h-th iteration can be written as: $f_{\text{SCA}} : \{\Theta_t^h, S_t\} \rightarrow \Theta_t^{(h+1)}$, and this process will be repeated as the Perceiver TF block repeats in order to maintain

the connection between Θ_t^h and the input S_t. The design of the cross-attention module is the key that significantly improves the computational scalability of Perceiver. For instance, our SCA results in $\mathcal{O}(FK)$, which is much cheaper than $\mathcal{O}(F^2)$ of the spectral Transformer in SpecTNT [4], given that K (dimension of the latent array) is typically small (i.e., $K \ll F$).

The *latent Transformer* module takes place after the SCA module. It contains a stack of N Transformers to perform standard self-attention on the latent arrays of Θ_t^h. The resulting complexity $\mathcal{O}(NK^2)$ is efficient as well. In the context of AMT, this process means the interactions among the onsets, pitches, and instruments are explicitly modeled. To perform multitrack AMT, we initialize K latent arrays and train each latent array to handle one specific task. Following [21], for an instrument, we arrange two latent arrays to model the onset and frame-wise (pitch) activations, respectively. This leads to $K = 2J$, where J is the number of target instruments.

The *temporal Transformer* module is placed to enable the communication between any pairs of Θ_t^h of different time steps. To make the *temporal Transformer* understand the time positions of each latent array, we add a trainable positional embedding to each Θ_t^0 during the initialization. Let $\theta_t^h(k)$, k = 0, . . . , K–1, denote each latent array in Θ_t^h, we arrange K parallel standard Transformers in which each serves the corresponding input sequence of latent arrays: $[\theta_0^h(k), \theta_1^h(k), \ldots, \theta_{T-1}^h(k)]$. The module is repeated M times, yielding a complexity of $\mathcal{O}(MT^2)$.

Finally, we repeat L times the Perceiver TF block to form the overall module. Note that, different from the original Perceiver, the weights of *spectral cross-attention* and *latent Transformer* are not shared across the repeated blocks.

3.3. Output Module

We utilize two GRU modules [22] with sigmoid activation function for the onset and frame-wise latent array outputs, respectively. We follow prior work [21] that uses the onset outputs to condition the frame-wise activation.

3.4. Multi-task Training Loss

We formulate the loss function for training the proposed model:

$$\mathcal{L} = \sum_{j=0}^{J-1} (l_{\text{onset}}^j + l_{\text{frame}}^j) \tag{1}$$

where l is the binary cross-entropy loss between the ground-truth and prediction, l_{onset}^j and l_{frame}^j are respectively the onset and frame activation losses for instrument j. Note that the losses for all J instruments should be computed, regardless of whether the corresponding instruments are active or not in a training sample. Therefore, a zero output is expected for instruments that are not present in the sample.

4. EXPERIMENTS

4.1. Datasets

We use four public datasets for evaluation. **Slakh2100** [23] contains 2100 pieces of multitrack MIDI and the corresponding synthesized audio. The MIDI files are a subset of Lakh dataset [24], and the audio samples were synthesized by professional-grade software. Instruments were grouped into 12 MIDI classes defined in the Slakh dataset.[1] We used the official train/validation/test splits in our experiments. **MAESTROv3** [25] contains about 200 hours of piano solo recordings with the aligned note annotations acquired by the MIDI capturing device on piano. We follow the official train/validation/test splits. **GuitarSet** [26] contains 360 high-quality guitar recordings and their synchronized note annotations. Since there is no official splits for this dataset, we follow the setting in [2]. The first two progressions of each style are used for training, and the last one is for testing. **MIR-ST500** [13] contains 500 Chinese-pop songs with note annotations for the lead vocal melody. We used the official train-test split. Although around 10% of the training set is missing due to failure links, we ensure the testing set is complete for fair comparison.

[1] There is no "Sound Effects", "Percussive" and "Ethnic" instruments. We grouped "Strings" and "Ensemble" into one instrument class.

4.2. Data Augmentations

Annotating data for multitrack AMT is labor intensive. To better exploit the data at hand, we apply two data augmentation techniques during training. Following previous works [4, 27], pitch-shifting is randomly performed to all the non-percussive instruments during training. We introduce the cross-dataset *random-mixing* (RM) technique. Let us first define three types of datasets:

- *Multi-track*: each sample contains multi-tracks of instrument-wise audio stems with polyphonic notes (e.g., Slakh), and no vocal signals are present.
- *Single-track*: each sample contains only a single non-vocal stem with polyphonic notes (e.g., MAESTRO and GuitarSet).
- *Vocal-mixture*: each sample is a full mixture of music with monophonic notes only for lead vocal (e.g., MIR-ST500). We employ a MSS tool [28] to separate each sample into vocal and accompaniment stems.

Each training sample is excerpted from a random moment of its original song with a duration depending on the model input length (e.g., 6 seconds). Suppose we want to transcribe J classes of instruments, and the corresponding instrument set is denoted as $\mathbf{\Omega} = \{\omega_j\}_{j=0}^{J-1}$. Then, we apply three treatments to the three mentioned types of datasets respectively as follows.

First, for a training sample s_i from a *multi-track* dataset, we denote its instrumentation template as $\boldsymbol{\mu}_i \subseteq \mathbf{\Omega}$, indicating the instruments present in s_i. Then, for each instrument ω_j in $\boldsymbol{\mu}_i$, it has a p% chance to be replaced by a ω_j in $\boldsymbol{\mu}_u$, where $i \neq u$ (i.e., a different sample). Second, for a sample s_i from a *single-track* dataset, we randomly pick an existing instrumentation template $\boldsymbol{\mu}_u$ ($i \neq u$) as its background. If the instrument of s_i is present in $\boldsymbol{\mu}_u$, that stem will be removed from $\boldsymbol{\mu}_u$. For instance, if s_i is a piano solo, then we will remove the piano stem from $\boldsymbol{\mu}_u$. From our preliminary experiment, presenting the solo example to model training without mixing it with a background can degrade the performance. Lastly, for a sample s_i from a *vocal-mixture* dataset, it has a q% chance to replace its background by two methods: (i) like the *single-track* treatment, we randomly pick an existing $\boldsymbol{\mu}_u$ ($i \neq u$) as its background; or (ii) we randomly pick an accompaniment stem separated from s_v, where $i \neq v$. For the second method, since the selected accompaniment stem does not have the ground-truth notes, we mask the instrument outputs and only count the loss for the vocal output (see Eq. 1).

4.3. Implementation Details

We implemented our system using PyTorch [29]. The audio waveform is re-sampled to 16kHz sampling rate. We set the model input length to be 6 seconds. The log-magnitude spectrogram is then computed using 2048 samples of Hann window and a hop size of 320 samples (i.e., 20 ms). The convolutional module contains 3 residual

Slakh	All	Piano	Bass	Drums	Guitar	Strings	Brass	Organ	Pipe	Reed	S.lead	S.pad	C.perc.
MT3†	.743	.780	.906	.773	.732	.551	.433	.363	.282	.440	.409	.234	.353
Ours (No-RM)	.763	.809	.921	.759	.727	.699	.632	.562	.578	.649	.677	.358	.458
Ours	**.798**	**.854**	**.930**	**.785**	**.777**	**.744**	**.732**	**.694**	**.666**	**.725**	**.769**	**.474**	**.575**

Table 2. The results of different models trained on (Mix) datasets and tested on Slakh2100. MT3† is our replication, as the instrument-wise results are not reported in [2]. "All" presents the Multi-instrument Onset F1 scores. The following columns show the Onset F1 scores for individual instrument. "S.lead", "S.pad", and "C.perc." stand for Synth Lead, Synth Pad, and Chromatic Percussion, respectively.

Dataset	Slakh	MAESTRO	GuitarSet	MIR-ST500
SpecTNT (Single)	-	**.969**	.907	.778
MT3 (Single)	.760	.960	.830	-
MT3 (Mix)	.760	.950	.900	-
MT3† (Mix)	.763	.958	.891	-
Ours (Single)	.808	.967	.903	.777
Ours (Mix+Vocal)	**.819**	.968	**.911**	**.785**
EFN	-	-	-	.666
JDC$_{note}$(L+U)	-	-	-	.697

Table 1. The results of Onset F1 scores. MT3† is our replication. Models with (Mix) or (Mix+Vocal) are trained on the mixture of datasets, while models with (Single) are trained on a single dataset.

blocks, each of them has 128 channels and is followed by an average pooling layer with a time-frequency filter of (1, 2).

For the Perceiver TF module, we use the following parameters (referring to Fig. 1): (i) depending on different experiment configurations, initialize $2J$ latent arrays, each uses a dimension of 128; (ii) stack $L = 3$ Perceiver TF blocks; (iii) for each Perceiver TF block, use 1 spectral cross-attention layer, $N = 2$ latent Transformer layers, and $M = 2$ temporal Transformer layers. All the Transformer layers has an hidden size of 128 with 8 heads for the multi-head attention. Finally, the output module is a 2-layer Bi-directional GRU with 128 hidden units. All of the Transformer module in the Perceiver TF include dropout with a rate of 0.15. The output dimension for onset and frame activations are 128 and 129, respectively, where 128 corresponds to the MIDI pitches, and the additional 1 dimension in the frame activation is for the silence. We use AdamW [30] as the learning optimizer. The initial learning rate and weight decay rate are set to 10^{-3} and 5×10^{-3}, respectively.

For final output, we take a threshold of 0.25 for both the onset and frame probability outputs to get the binary representations, so the frame-wise activations can be merged to generate each note in a piano-roll representation. No further post-processing is applied.

For data augmentation, all of the non-percussive instruments of a training example have a 100% probability to be pith-shift up or down by at most 3 semi-tones. For random-mixing, we use $p = 25\%$ and $q = 50\%$ for data from *multi-track* and *vocal-mixture* datasets, respectively. To generate an input sample, all the instrument stems in each training example are linearly summed up.

4.4. Baselines

Two state-of-the-art models, MT3 [2] and SpecTNT [4], are selected as the baselines. For MT3, we replicated the model following [2],[2] which includes the official model checkpoint and inference pipeline on the test set. For SpecTNT, we adopted the configuration used for vocal melody extraction reported in [4]. In the preliminary experiments, we found it non-trivial to successfully train the original SpecTNT on Slakh2100 under the multi-instrument setting, so we skip this experiment. For vocal transcription, the best results of EFN [13] and JDC$_{note}$(L+U) [14] are reported.

[2]https://github.com/magenta/mt3/blob/main/mt3/colab/music_transcription_with_transformers.ipynb

4.5. Evaluation Metrics

We use "Onset F1" score, which indicates the correctness of both pitches and onset timestamps, as the evaluation metric for comparison with previous work [2]. To further evaluate the performance of multi-instrument transcription, we report the "Multi-instrument Onset F1" score for the Slakh dataset. The outputs from our replicated MT3 model are grouped into 12 instrument classes based on their program numbers. The Multi-instrument Onset F1 score we used only counts Onset F1, which is similar to the MV2H metric [31]. It could be slightly different from the one used in [2], since the "Drums" outputs do not contain clear offset information.

4.6. Result and Discussion

Table 1 shows the comparison in terms of Onset F1 between the proposed model and baselines. The proposed model and SpecTNT which directly model the spectral inputs with the attention mechanism shows higher performance for cases even trained on low resources of a single dataset, such as GuitarSet. On MIR-ST500, the proposed model significantly outperforms the baselines. Although SpecTNT (Single) performs slightly better than our model on MAESTRO, we still consider Perceiver TF to be more advantageous to practical use for its better inference efficiency.

Table 2 presents the Multi-instrument Onset F1 (instrument-weighted average) and the Onset F1 scores of individual instrument classes on Slakh2100 to reveal instrument-wise performance. Compared to MT3†, our model without the random-mixing augmentation (No-RM) performs significantly better on less-common instruments such as "Pipe" (the Onset F1 score is upper by over 100%). Applying random-mixing in training can further boost the performance in all cases, indicating the technique indeed improves the model robustness to discriminate between different instruments. Finally, we observe that combining multi-instrument and vocal transcriptions can improve the vocal transcription alone, as the combined model is trained with more randomly mixed vocal-accompaniment samples.

5. CONCLUSION

We have presented Perceiver TF, a novel architecture that adequately addresses the *model scalability* problem for multitrack AMT. To address the *instrument discrimination* issue, we have proposed the random-mixing augmentation technique, which significantly facilitates the data usability across datasets. Our system has demonstrated state-of-the-art performance on different public datasets. We believe Perceiver TF is generic and can be applied to other analogous tasks.

6. REFERENCES

[1] Kin Wai Cheuk, Dorien Herremans, and Li Su, "Reconvat: A semi-supervised automatic music transcription framework for low-resource real-world data," in *Proc. ACM Multimedia*, 2021, pp. 3918–3926.

[2] Josh Gardner, Ian Simon, Ethan Manilow, Curtis Hawthorne, and Jesse Engel, "MT3: Multi-task multitrack music transcription," in *Proc. ICLR*, 2021.

[3] Andrew Jaegle, Felix Gimeno, Andy Brock, Oriol Vinyals, Andrew Zisserman, and Joao Carreira, "Perceiver: General perception with iterative attention," in *Proc. ICML*, 2021, pp. 4651–4664.

[4] Wei-Tsung Lu, Ju-Chiang Wang, Minz Won, Keunwoo Choi, and Xuchen Song, "SpecTNT: A time-frequency transformer for music audio," in *Proc. ISMIR*, 2021.

[5] Stefan Uhlich, Marcello Porcu, Franck Giron, Michael Enenkl, Thomas Kemp, Naoya Takahashi, and Yuki Mitsufuji, "Improving music source separation based on deep neural networks through data augmentation and network blending," in *Proc. ICASSP*, 2017, pp. 261–265.

[6] Xuchen Song, Qiuqiang Kong, Xingjian Du, and Yuxuan Wang, "Catnet: Music source separation system with mix-audio augmentation," *arXiv preprint arXiv:2102.09966*, 2021.

[7] Zafar Rafii, Antoine Liutkus, Fabian-Robert Stöter, Stylianos Ioannis Mimilakis, Derry FitzGerald, and Bryan Pardo, "An overview of lead and accompaniment separation in music," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 26, no. 8, pp. 1307–1335, 2018.

[8] Yun-Ning Hung, Yi-An Chen, and Yi-Hsuan Yang, "Multi-task learning for frame-level instrument recognition," in *Proc. ICASSP*, 2019, pp. 381–385.

[9] Andreas Jansson, Rachel M Bittner, Sebastian Ewert, and Tillman Weyde, "Joint singing voice separation and f0 estimation with deep u-net architectures," in *Proc. EUSIPCO*, 2019, pp. 1–5.

[10] Yu-Te Wu, Berlin Chen, and Li Su, "Multi-instrument automatic music transcription with self-attention-based instance segmentation," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 2796–2809, 2020.

[11] Keitaro Tanaka, Takayuki Nakatsuka, Ryo Nishikimi, Kazuyoshi Yoshii, and Shigeo Morishima, "Multi-instrument music transcription based on deep spherical clustering of spectrograms and pitchgrams," in *Proc. ISMIR*, 2020.

[12] Kin Wai Cheuk, Keunwoo Choi, Qiuqiang Kong, Bochen Li, Minz Won, Amy Hung, Ju-Chiang Wang, and Dorien Herremans, "Jointist: Joint learning for multi-instrument transcription and its applications," *arXiv preprint arXiv:2206.10805*, 2022.

[13] Jun-You Wang and Jyh-Shing Roger Jang, "On the preparation and validation of a large-scale dataset of singing transcription," in *Proc. ICASSP*, 2021, pp. 276–280.

[14] Sangeun Kum, Jongpil Lee, Keunhyoung Luke Kim, Taehyoung Kim, and Juhan Nam, "Pseudo-label transfer from frame-level to note-level in a teacher-student framework for singing transcription from polyphonic music," in *Proc. ICASSP*, 2022.

[15] Jui-Yang Hsu and Li Su, "Vocano: A note transcription framework for singing voice in polyphonic music.," in *Proc. ISMIR*, 2021, pp. 293–300.

[16] Anmol Gulati, James Qin, Chung-Cheng Chiu, Niki Parmar, Yu Zhang, Jiahui Yu, Wei Han, Shibo Wang, Zhengdong Zhang, Yonghui Wu, et al., "Conformer: Convolution-augmented transformer for speech recognition," in *Proc. INTERSPEECH*, 2020.

[17] M. Won, K. Choi, and X. Serra, "Semi-supervised music tagging transformer," in *Proc. ISMIR*, 2021, pp. 769–776.

[18] Yun-Ning Hung, Ju-Chiang Wang, Xuchen Song, Wei-Tsung Lu, and Minz Won, "Modeling beats and downbeats with a time-frequency transformer," in *Proc. ICASSP*, 2022, pp. 401–405.

[19] Ju-Chiang Wang, Yun-Ning Hung, and Jordan B. L. Smith, "To catch a chorus, verse, intro, or anything else: Analyzing a song with structural functions," in *Proc. ICASSP*, 2022, pp. 416–420.

[20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun, "Identity mappings in deep residual networks," in *European conference on computer vision*. Springer, 2016, pp. 630–645.

[21] Curtis Hawthorne, Erich Elsen, Jialin Song, Adam Roberts, Ian Simon, Colin Raffel, Jesse Engel, Sageev Oore, and Douglas Eck, "Onsets and frames: Dual-objective piano transcription," in *Proc. ISMIR*, 2018, pp. 50–57.

[22] Junyoung Chung, Caglar Gulcehre, KyungHyun Cho, and Yoshua Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," in *Proc. NeurIPS*, 2014.

[23] Ethan Manilow, Gordon Wichern, Prem Seetharaman, and Jonathan Le Roux, "Cutting music source separation some Slakh: A dataset to study the impact of training data quality and quantity," in *Proc. WASPAA*. IEEE, 2019.

[24] Colin Raffel, "Learning-based methods for comparing sequences, with applications to audio-to-midi alignment and matching," 2016, Columbia University.

[25] Curtis Hawthorne, Andriy Stasyuk, Adam Roberts, Ian Simon, Cheng-Zhi Anna Huang, Sander Dieleman, Erich Elsen, Jesse Engel, and Douglas Eck, "Enabling factorized piano music modeling and generation with the maestro dataset," in *Proc. ICLR*, 2019.

[26] Qingyang Xi, Rachel M Bittner, Johan Pauwels, Xuzhou Ye, and Juan Pablo Bello, "Guitarset: A dataset for guitar transcription.," in *Proc. ISMIR*, 2018, pp. 453–460.

[27] Sangeun Kum and Juhan Nam, "Joint detection and classification of singing voice melody using convolutional recurrent neural networks," *Applied Sciences*, vol. 9, no. 7, pp. 1324, 2019.

[28] Qiuqiang Kong, Yin Cao, Haohe Liu, Keunwoo Choi, and Yuxuan Wang, "Decoupling magnitude and phase estimation with deep resunet for music source separation.," in *Proc. ISMIR*, 2021.

[29] Paszke et al., "Pytorch: An imperative style, high-performance deep learning library," in *Neural Information Processing Systems*, 2019, vol. 32.

[30] Ilya Loshchilov and Frank Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2017.

[31] Andrew McLeod and Mark Steedman, "Evaluating automatic polyphonic music transcription.," in *Proc. ISMIR*, 2018, pp. 42–49.